

Binary Search Tree

****You do not need to write the BSTNode class, inorder()/insert()/delete()/balance() etc. functions unless specified in the problem.****

****Just design the function specified in the problem.****

****No need to write driver code.****

****You may create helper functions to solve tasks****

****This document is a compilation of previous semester's exam (midterm/final) problems and lab quiz problems. Try to practice relevant problems from online judges as well.****

1.

Tasks from Lab 7 - [W Lab 7 - Binary Search Tree.docx](#)

Tasks from Book - [PDF Practice Sheet 9 - Tree.pdf](#)

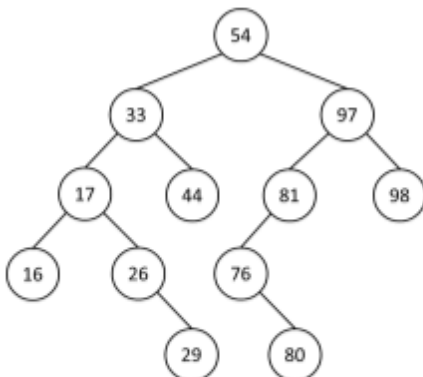
2.

Given the **array representation** of a binary tree: [None value means the node is empty]
[None, 19, 22, 10, 32, 20, None, 30, None, 5, None, 7, None, None, 2, -2]

- Draw** the binary tree. [2 marks]
- Write** the post-order and pre-order traversal sequence of the tree. [1 marks]
- Use the **pre-order** traversal sequence in part b to **insert** the elements in that order in an initially empty binary search tree, and show the resulting binary search tree. Note: Consider the first element of the pre-order sequence as the root. [2 marks]
- Perform the following operations **step by step sequentially** on the Binary Search Tree you created in part c.
i) Delete node 5 with the help of its successor. [2 marks]
ii) Delete node 20 with the help of its predecessor.

3.

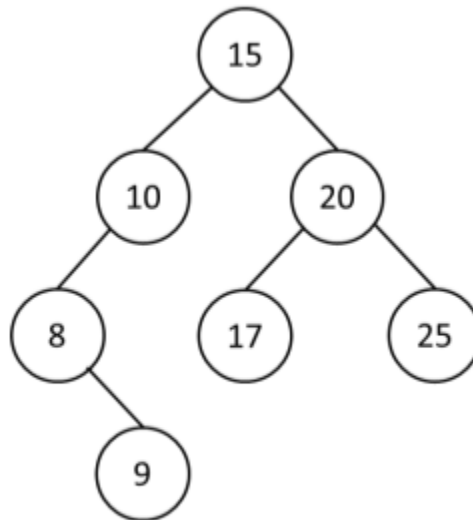
Take a look at the following binary search tree. Do the following operations step by step:



- Traverse the given tree in pre-order and write the output in your script. [2]
- Insert the key 89 in the given BST and draw the BST. [3]
- Is the modified tree a balanced binary tree? (Yes/No) [2]
- We know that the in-order traversal of BST provides an ascending order of the elements. Can you propose an order of traversal that will print the BST in descending order without using any other data structure? Apply it and print the elements. [3]

4.

Consider the following BST.



- A. Delete the root 15 with its predecessor and draw the resulting tree.
- B. Is the tree drawn on A a complete binary tree (Yes / No)?
- C. Is the tree drawn on A a balanced binary tree (Yes / No)?

5.

Question:

Write a recursive function **find_elements_leq()** that takes a BST and a target number, and returns a **string** with all BST elements **less than or equal** to the target. Use **preorder traversal** to populate the string.

Note: You cannot use any loops or global variables. You may use any number of helper functions as needed. Assume the `TreeNode` class is provided, where each node contains an integer value in the element attribute and pointers to the left and right child nodes.

Sample Input	Sample Output
<pre> 10 / \ 5 15 / \ 2 7</pre> Target = 10	10 5 2 7
<pre># Python Function Signature def find_elements_leq(root, target, result=None) # Python Function Call print(find_elements_leq(root, target))</pre>	
<pre>//Java Method Signature public static String find_elements_leq(TreeNode root, int target) //Java Method Call System.out.println(find_elements_leq(root, target));</pre>	

6.

Question:

Write a recursive function `find_elements_geq()` that takes a BST and a target number, and returns a **string** with all BST elements **greater than or equal** to the target. Use **preorder traversal** to populate the string.

Note: You cannot use any loops or global variables. You may use any number of helper functions as needed. Assume the `TreeNode` class is provided, where each node contains an integer value in the `element` attribute and pointers to the left and right child nodes.

Sample Input	Sample Output
<pre> 10 / \ 6 16 \ / 8 13</pre>	<pre>10 16 13</pre>
Target = 10	
<pre># Python Function Signature def find_elements_geq(root, target, result=None) # Python Function Call print(find_elements_geq(root, target))</pre>	
<pre>//Java Method Signature public static String find_elements_geq(TreeNode root, int target) //Java Method Call System.out.println(find_elements_geq(root, target));</pre>	

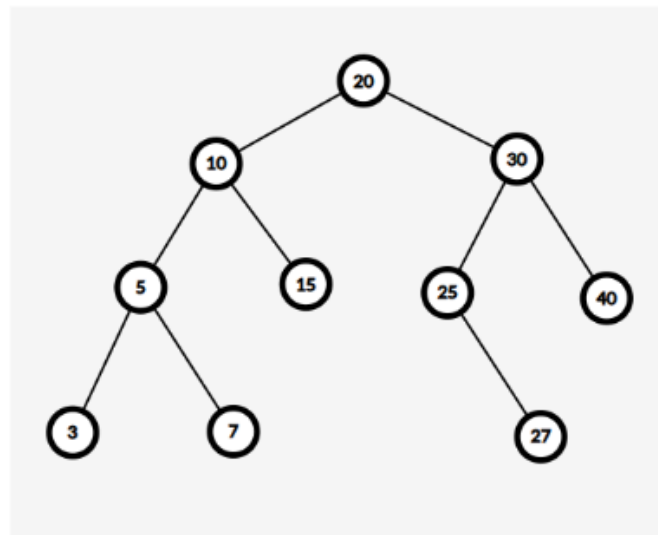
7.

Given the root of the following Binary Search Tree and an integer k , return the product of all node values that are strictly greater than k .

Note:

- Use **recursion**.
- If **no nodes** are greater than k , return 1.
- The tree will have **distinct positive integers**.
- You may define and use **helper functions**.

Tree:



Sample Input	Sample Output
15	$20 \times 25 \times 27 \times 30 \times 40 = 16200000$ Explanation: Nodes > 15
25	$27 \times 30 \times 40 = 32400$ Explanation: Nodes > 25
35	$40 = 40$ Explanation: Only node > 35
100	1 Explanation: No nodes > 100

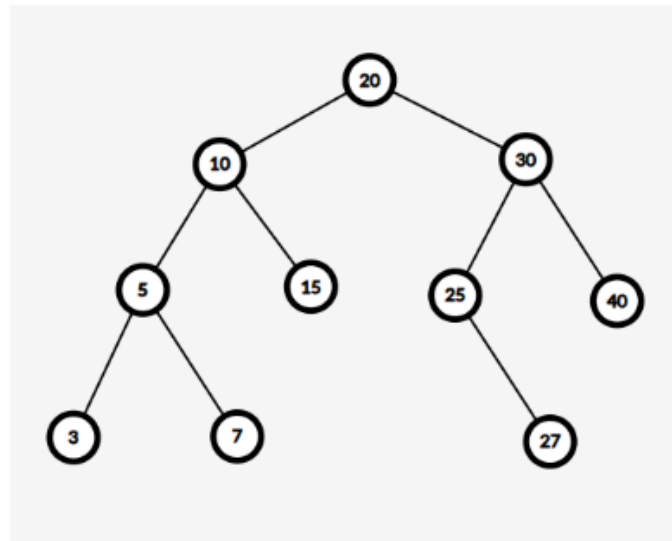
8.

Given the root of the same Binary Search Tree and an integer target, **count the number of nodes whose value is divisible by target.**

Note:

- Use **recursion**.
- If **no nodes** are divisible by the target, return 0.
- Assume all values in the tree are **positive integers**.

Tree:



Sample Input	Sample Output
5	5 Explanation: 5, 10, 20, 25, 30
3	3 Explanation: 3, 15, 27
7	1 Explanation: Only 7
13	0 Explanation: No node is divisible by 13

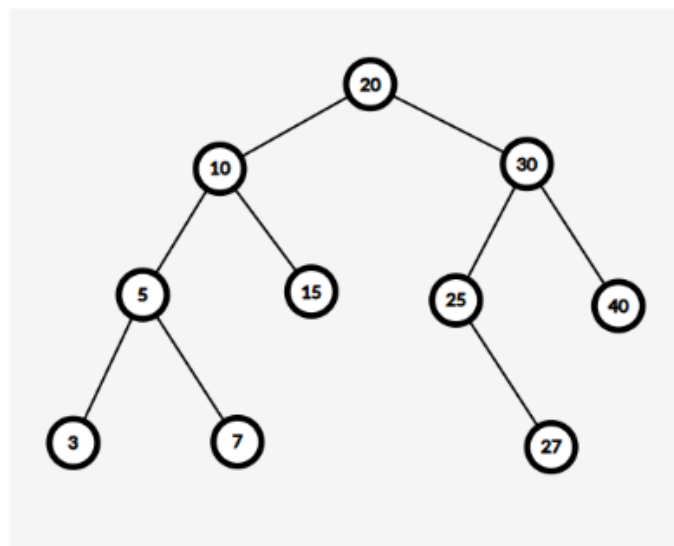
9.

Given the root of a Binary Search Tree and two node values x and y, find the nodes in the shortest path between the given two nodes.

Note:

- If either node is not found in the BST, return "No Path Found".
- For python, [can](#) use array, reverse and pop if needed.
- For java, can use ArrayList, reverse and remove if needed. Otherwise return as string.
- Solve the problem using recursion.
- Can use as many helper functions you want.

Tree:



Sample Input	Sample Output
path_between_nodes(root, 5, 15)	[5, 10, 15]
path_between_nodes(root, 3, 27)	[3, 5, 10, 20, 30, 25, 27]
path_between_nodes(root, 10, 10)	[10]
path_between_nodes(root, 7, 99)	No Path Found

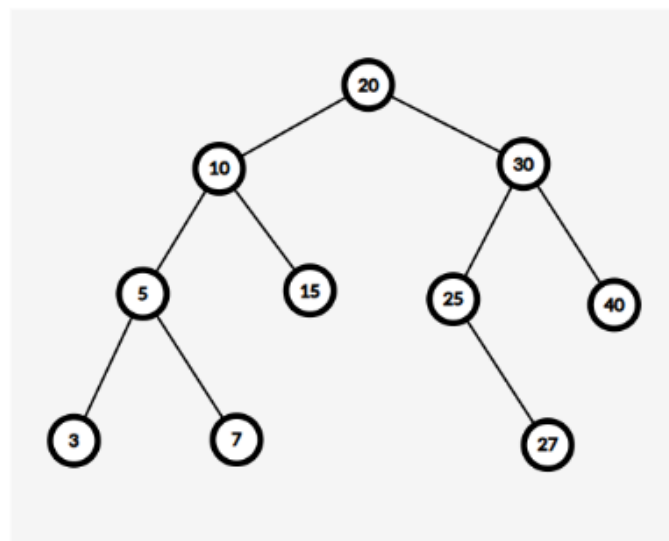
10.

Given the root of a Binary Search Tree and two node values x and y , find the number of nodes in the shortest path between the given two nodes.

Note:

- If either node is not found in the BST, return -1.
- For the same node ($x == y$), return 1.
- Solve the problem using recursion.
- Can use as many helper functions you want.
- For python and java, can use array, ArrayList, reverse and pop / remove if needed.

Tree:



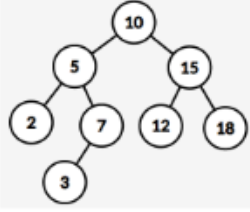
Sample Input	Sample Output
distance_in_nodes(root, 5, 15)	3 Explanation: path is 5 -> 10 -> 15
distance_in_nodes(root, 3, 27)	7 Explanation: path is 3 -> 5 -> 10 -> 20 -> 30 -> 25 -> 27
distance_in_nodes(root, 10, 10)	1 Explanation: Same node — path has only one node 10
distance_in_nodes(root, 7, 99)	-1 Explanation: Node not found

11.

Mirror, Mirror on the Tree: Is it Even or the free?

Write a function **mirror_parity()** that takes the root of a Binary Search Tree (BST) and a single node x as parameters. The function should find the mirror node corresponding to node x and return "**Even**" if the mirror node's value is even, or "**Odd**" if the value is odd. If the mirror node does not exist, return "**No Mirror Found**".

Only recursive solution allowed. You can use helper functions.

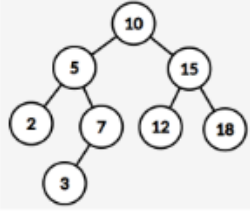
Example of BST	Sample Inputs	Sample Outputs	Explanation
	mirror_parity(root, 5)	Odd	Mirror of 5 = 15 15 is Odd
	mirror_parity(root, 7)	Even	Mirror of 7 = 12 12 is Even
	mirror_parity(root, 3)	No Mirror Found	No Mirror Node of 3

12.

Mirror, Mirror, multiply and see: what's the Answer for the tree?

Write a function **mirror_multiply()** that takes the root of a Binary Search Tree (BST) and a single node x as parameters. The function should find the mirror node corresponding to node x and return the **multiplication of the given node's value and its mirror node's value** if a mirror node is found. If a mirror node does not exist, return "**No Mirror Found**".

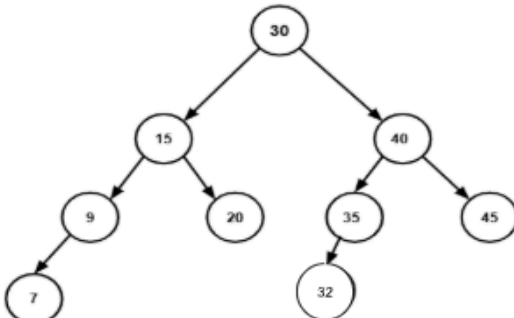
Only recursive solution allowed. You can use helper functions.

Example of BST	Sample Inputs	Sample Outputs	Explanation
	mirror_multiply(root, 5)	75	Mirror of 5 = 15 $5 \times 15 = 75$
	mirror_multiply(root, 7)	84	Mirror of 7 = 12 $7 \times 12 = 84$
	mirror_multiply(root, 3)	No Mirror Found	No Mirror Node of 3

13.

Question: Luffy is playing an online game where each city is a node in a Binary Search Tree (BST), holding an energy cost. Starting from the root, Luffy must conquer cities along the path to his target. The total energy is the product of all node values on the route. If the target exists, the energy is shown; otherwise, the system says "route does not exist."

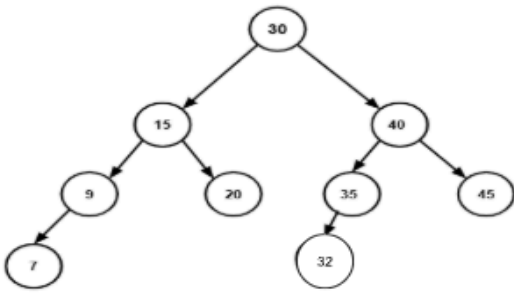
Now, write a function **calculate_energy(root, destination)** that takes the root of the BST and the target destination value as parameters and returns the calculated energy or "route does not exist" if the destination is not found.

Sample Input	Sample Output
calculate_energy(root,35) 	Product = 42000 Explanation: Route : 30→40→35 So, product = 30*40*35=42000

14.

Question: Levy is playing an online game where each city is a node in a Binary Search Tree (BST), holding a visiting fare. Starting from the root, Levy must conquer cities along the path to his target. The total visiting fare is the sum of all node values on the route. If the target exists, the total fare is shown; otherwise, the system says "route does not exist."

Now, write a function **calculate_fare(root, destination)** that takes the root of the BST and the target destination value as parameters and returns the total visiting fare or "route does not exist" if the destination is not found.

Sample Input	Sample Output
calculate_fare(root,35) 	Total visiting fare = 105 Explanation: Route : 30→40→35 So, product = 30+40+35=105

15.

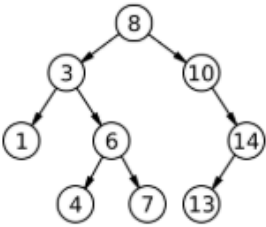
Question:

Given the **root** of a Binary Search Tree and an integer **k**, write a function **smallest_sum(root, k)** that returns the **sum of the k smallest nodes** of the BST.

You can assume **k** is **always valid** ($0 \leq k \leq$ total number of nodes).

Constraints:

- You must solve it using only recursion.
- No extra data structures like arrays, lists, or stacks.

Example of BST	Sample Inputs	Sample Outputs	Explanation
	smallest_sum(root, 3)	8	Sum 3 smallest nodes are: $1+3+4 = 8$
	smallest_sum(root, 5)	21	Sum 5 smallest nodes are: $1+3+4+6+7 = 21$
	smallest_sum(root, 6)	29	Sum 6 smallest nodes are: $1+3+4+6+7+8 = 29$

16.

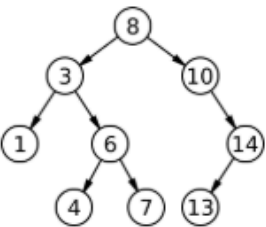
Question:

Given the **root** of a Binary Search Tree and an integer **k**, write a function **smallest_mul(root, k)** that returns the **multiplication of the k smallest nodes** of the BST.

You can assume **k** is **always valid** ($0 \leq k \leq$ total number of nodes).

Constraints:

- You must solve it using only recursion.
- No extra data structures like arrays, lists, or stacks.

Example of BST	Sample Inputs	Sample Outputs	Explanation
	smallest_mul(root, 3)	12	Sum 3 smallest nodes are: $1*3*4 = 12$
	smallest_mul(root, 4)	72	Sum 4 smallest nodes are: $1*3*4*6 = 72$
	smallest_mul(root, 6)	4032	Sum 6 smallest nodes are: $1*3*4*6*7*8 = 4032$

17.

Suppose you have a set of 10 vertices which are denoted as A, B, C, D, E, F, G, H, I, J.
The matrix representation below shows the relationship between any of these two vertices.

	A (0)	B (1)	C (2)	D (3)	E (4)	F (5)	G (6)	H (7)	I (8)	J (9)
A (0)		5	3							
B (1)				7						
C (2)				4	6					
D (3)					2					
E (4)										
F (5)							8			
G (6)								9		
H (7)									10	
I (8)										
J (9)										11

[Note: Vertex A is mapped as index position 0, Vertex B is mapped as index position 1 in the matrix and so on]

- Using the above matrix representation, draw an adjacency list representation of the above nodes. [3]
- Determine** whether the graph is directed or undirected. If it is undirected, transform it into a directed graph and if it is directed, transform it into an undirected one using the above matrix representation. Write down the resulting matrix in your script [1+2]
- Suppose that the vertices A to J have the following weights 31, 19, 25, 40, 45, 35, 33, 27, 14, 21, 37 respectively.
 - Draw a Binary Search Tree using the vertices A to J sequentially based on their weights. Write down the node's weights while drawing the BST. [2]
 - Delete the node with the key of value 31 with the help of its predecessor of the above tree and draw the new tree. [2]

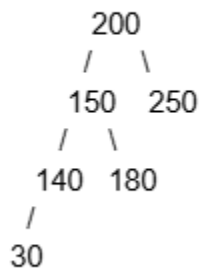
18.

Question 2 : Answer the following short questions:

- a) What is the time complexity of insertion, deletion and searching when a BST is skewed (all nodes having only left children or only right children)?
- b) Given the representation of a binary tree: [null, 25, 40, 60, 80, 90, 120], what is the left and right child of 60 (index 3)?
- c) You are given two sequences:
 - Inorder Traversal: [2, 5, 7, 10, 12, 15, 20]
 - Preorder Traversal: [10, 5, 2, 7, 15, 12, 20]Construct the tree from the two given sequences.

19.

- 1) What output will be returned from the maxTreasure function for this tree?



- 2) Why are Binary Search Trees (BSTs) generally more efficient than ordinary Binary Trees?

- 3) If a Binary Tree has 31 nodes, what is the minimum possible height (root at height 0)?

20.

You are given the root of a **Binary Search Tree (BST)**. Each node stores a certain number of coins. A traveler starts at the **root** and moves down to a **leaf**, collecting coins from each node along the way.

Your task is to write a **recursive function** `maxTreasure(root)` that will:

1. Find the **maximum sum of coins** that can be collected from the root to any leaf.
2. Return the **path of node values** that leads to this maximum sum as a string.

Constraints:

- You can use Global or Instance variables if you want
- Use **recursion only** (no loops).
- Do **not** use built-in functions like `max()` or `Math.max()`.
- Do **not** use arrays/lists to store all possible sums;
- Build the **single best path as a string** while recursing.
- Assume all node values are positive integers.

Sample Input	Sample Output
<pre> 20 / \ 10 30 / \ \ 5 15 40</pre> <pre>def maxTreasure(root): # return (path_str) pass</pre> <pre>static class Result { int sum; String path; Result(int s, String p) { sum = s; path = p; } } public static Result maxTreasure(Node root) { // write your code here }</pre>	Output: "20->30->40" Explanation: Paths are: 20 → 10 → 5 = 35 20 → 10 → 15 = 45 20 → 30 → 40 = 90 (maximum) So the maximum treasure is 90, and the best path is "20->30->40".

Hint: The maximum path does not always lie in the right subtree.

21.

Simulate what happens if we delete the following nodes from the BST given below. Consider each case independently. **Here, no need to balance the BST.**

- I. Delete Node **9**
- II. Insert Node **10**

